

Class 4

Deploy Backend and DBMS

*Today you put something on the internet that other people can load.
Almost everything that goes wrong today goes wrong the same way in every class after this one.*

Where we have got to

THE SHAPE OF THIS COURSE

- Class 3 — a web server, handing out files that already exist. It could not accept anything.
- Today — the program that accepts something, and the database that keeps it.
- Next week — the halves joined, on one address, with the doors you opened closed again.
- Nothing today is a new idea. It is class 1's packages and class 2's services, pointed at something real.

You are still deploying software you did not write. Today it has a database behind it, which changes nothing about your job and everything about the consequences of getting it wrong.

Two applications, and why

HOW TODAY WORKS

- We deploy a NEW application together in this room: [school](#), a student register
- Afterwards, on your own, you do the whole procedure again on [your noticeboard](#). That one is lab04 and it carries the mark.
- Different names throughout — different directory, database, user, unit file and port. Nothing is shared, on purpose.
- Today's sheet explains everything. lab04 explains almost nothing, and that is the point of it.

Doing it once with me and once alone is the whole design. The second time is where it stops being copying.

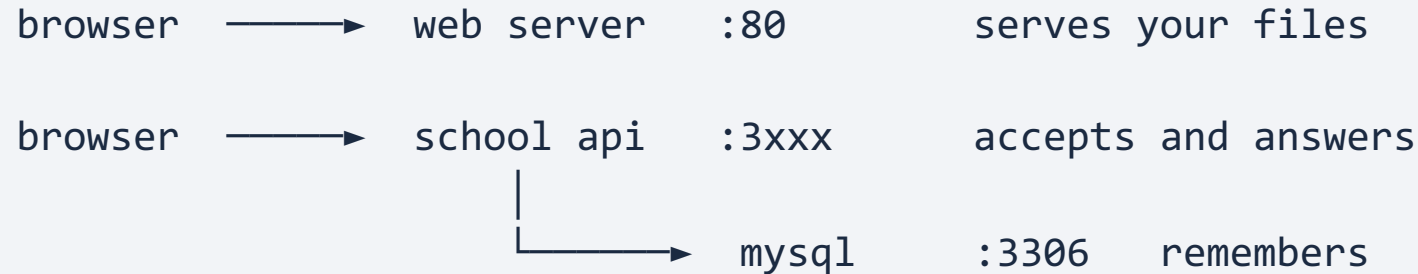
1

A database is a server

Not a file your program opens. Not a library it imports. A separate program, with its own users, that your application is merely a client of.

Three servers on one machine

PART 1 · WHAT YOU ARE BUILDING



- Three processes, three ports, three separate ideas of who you are.
- Being `sysadmin` here makes you nobody in particular to the database
- All three can be running perfectly while two of them cannot talk to each other
- So the first question in any failure is always: which of the three?

`systemctl status` and `journalctl -u` work the same way on all three. You are not learning new tools today.

The address column, again

PART 1 · LISTENING, AND TO WHOM

```
$ ss -tln
State  Recv-Q  Send-Q      Local Address:Port
LISTEN  0        511         0.0.0.0:80      ← the world
LISTEN  0        151         127.0.0.1:3306 ← this machine
```

- **0.0.0.0** — every address this machine has, including the one the world reaches.
- **127.0.0.1** — this machine and nothing else. Those packets never touch a network card.
- Class 3 taught you this column. Today two services on one machine answer it differently.
- So: does the database need a firewall rule of its own? Work out the answer, do not take mine.

A database reachable from the internet is one of the most common ways an organisation loses everything at once — and it is one line of configuration away.

Predict, then go and find out

PART 1 · DO NOT SKIP THIS

```
# package: mysql-server      service: mysql      version: must be 8

$ systemctl is-enabled mysql ; systemctl is-active mysql
```

- Does a freshly installed database start itself? Class 3's web server did not. Commit to an answer before you look.
- Two packagers, two different decisions, and neither of them told you. *You look.*
- Installing it is class 1's command — the sheet gives you the package name and not the line. Check the version too: it is the first thing anyone asks when a thing misbehaves.

Ten seconds, one command, and a fact you will not forget. Watching me type it is worth nothing.

Two kinds of administrator

PART 1 · IDENTITY

```
$ sudo mysql -e "SELECT VERSION();"      # no password asked for

$ mysql -e "SELECT 1;"                   # as yourself
ERROR 1045 (28000): Access denied for user 'sysadmin'@'localhost'
```

- The administrator account is tied to the **operating system user**, not to a password. Being root here is the whole credential.
- **Convenient for you at a terminal. Useless to a program, and that is deliberate**
- So your application needs an identity of its own — which is Part 2

Read the refusal word for word. `1045` and `28000` are the two numbers you will be searching for at some point in the next three years.

2

A user that owns only its own data

Not politeness. An application that can only touch its own data cannot destroy anything else when it is wrong — or when somebody else is driving it.

PART 2 · LEAST PRIVILEGE

```
# db/ holds THREE scripts. This is the only one needing the administrator.  
CREATE DATABASE school CHARACTER SET utf8mb4;  
CREATE USER 'school'@'localhost' IDENTIFIED BY RANDOM PASSWORD;  
GRANT ALL PRIVILEGES ON school.* TO 'school'@'localhost';  
$ sudo mysql < setup.sql          # read it BEFORE you run it
```

- A script, not typing — readable before it runs, reviewable by somebody who is not you, identical on the next machine.
- `'school'@'localhost'` is ONE identity — the name AND where it may connect from.*
- `school.*` is the scope: that one database, and nothing outside it.
- `utf8mb4` stores Thai and emoji; the older utf8 cannot. Already the default here — write it anyway, because a default is a file somebody can edit.

Then ask the server what that user may do, connect as it, and look around. What you can and cannot see is the grant, made visible.

Let the server choose the password

PART 2 · A CREDENTIAL, NOT A PASSWORD

```
$ sudo mysql < setup.sql
user      host      generated password    auth_factor
school    localhost 20 characters, once    1

# tab-separated, not a drawn table: mysql is reading a FILE, not a terminal
```

- Shown once. The server keeps a hash and genuinely cannot tell you again.
- A password you invent for a service is one you will reuse, or shorten so you can remember it. Nobody memorises this one.
- It goes in exactly one file on your VM. Not in your answer file, not in a chat message, not in git.

No question in this course will ever ask you for it, or for any part of it, or for which characters it contains. If something does ask, it is not from me.

One password, one file

PART 2 · A CREDENTIAL IS CONFIGURATION

```
$ install -m 600 /dev/null /srv/school/db/school.env # private FIRST
SCHOOL_USER=school SCHOOL_DATABASE=school SCHOOL_PASSWORD=...

$ set -a; . /srv/school/db/school.env; set +a
$ MYSQL_PWD="$SCHOOL_PASSWORD" mysql -u school -e "SHOW DATABASES;"
```

- **KEY=value**, one per line — the same shape as `.env`, systemd's `EnvironmentFile=`, and Compose in class 8.
- A prefix assignment: the password is in THAT command's environment and nowhere else — never on a command line, which `ps` shows to everyone.
- Now `SHOW GRANTS` and `SHOW DATABASES` as that user: the scope, from the inside.

Mode 600 before the password is in it, not after. `chmod` afterwards leaves a window, and windows are what get exploited.

Why the scope is the point

PART 2 · BLAST RADIUS

- The question is not “what can this user do?” but *“what happens when it is not you driving?”
- With an administrator's credentials, a break-in hands over **every** database on the server
- Scoped, the same break-in hands over one application's data.
- The credential sits in a file on the machine. Files get read. Plan for that day.
- You meet this scope again in an hour, from the other side: a command that fails BECAUSE you got it right.

Do not widen a grant to make an error go away. Find out what wanted the privilege, and whether it should have it.

Shape and contents are different jobs

PART 2 · WHO OWNS WHAT

```
$ MYSQL_PWD="$SCHOOL_PASSWORD" mysql -u school school \  
    -e "source schema.sql"          # the SHAPE: table, columns, key  
  
$ MYSQL_PWD="$SCHOOL_PASSWORD" mysql -u school school \  
    -e "source seed.sql"            # the CONTENTS: three people
```

- Two files because they are two decisions. The shape changes when the code does; the contents change all day.
- `source` is the CLIENT's command, `<` is the SHELL's — `< schema.sql` does the same job, but only `source` names the file when line 2 fails.
- Neither one needed `sudo`. That is the grant doing its job — a boundary you worked inside, not one you worked around.

Two commands and not one: repeated `-e` is CONCATENATED, so a single line with both would send mysql looking for a file called ``schema.sql source seed.sql``. And read `seed.sql` before running it — it opens with a `DELETE`.

Looking at it from your laptop

PART 2 · WORKBENCH OVER SSH

```
Connection Method:  Standard TCP/IP over SSH
SSH Hostname:       lvm#####.sit.kmutt.ac.th:22    ← your VM
MySQL Hostname:     ?                               ← from WHERE?
MySQL Server Port:  3306                            Username: school
```

- The database is not on the network. So how does anyone ever look at it?
- You do not expose it. You reach it through a channel you already have — the same ssh you log in with.
- Two hostnames in one dialog and they are not the same machine. Work out which is which before you type.
- Every production database you will ever meet is reached exactly like this.

Workbench you already know from your database course. What is new is the field asking where the database is FROM THE SERVER'S point of view — and that the table you built two minutes ago with a script is sitting there, with rows in it, when you arrive.

3

Dependencies and schema

A program does not arrive alone. It arrives with a list of things it needs, and a description of the tables it expects, and it will not run until both are dealt with.

Three commands, one of them new

PART 3 · GETTING THE PIECES ONTO THE MACHINE

```
# Node from the vendor's repository – key, list file, install
$ curl -fsSL .../nodesource-repo.gpg.key | sudo gpg --dearmor -o ...

$ sudo install -d -o sysadmin -g sysadmin -m 2775 /srv/school # NEW
$ tar -xzf .../school.tar.gz -C /srv/school --strip-components=1 # no sudo
$ find /srv/school -perm -u+w -exec chmod g+w {} + # the w setgid cannot give
```

- Class 3's three moves for a vendor repository. Ubuntu's own nodejs is 18, unsupported since April 2025.
- `install -d = mkdir + chown + chmod`, in one — so the directory is never wrong, not even for an instant
- `--strip-components=1` drops the archive's top directory*; the `find` adds the `w setgid` never hands down.

Only `install` is new — and Part 2 uses it again, on a file with a password in it. The other two are class 3's, here so nobody discovers at 3pm that class 3 was the last time anybody explained them.

Install exactly what was tested

PART 3 · REPRODUCIBILITY

```
$ npm install      # install dependencies – and UPDATE the record
```

```
$ npm ci          # install dependencies EXACTLY as recorded
```

```
npm error code EUSAGE    ← what you get with no package-lock.json
```

- Your application depends on ~105 packages. Most it never asked for — they are what its dependencies depend on.
- `npm install` may quietly give you versions nobody has tested. `npm ci` refuses to guess.
- `package-lock.json` records the exact version of every one of them, including the ones you never chose.
- You want the versions that were tested, not the ones that exist this afternoon.

This is the single cheapest good habit in the whole course. One extra character, and your deployment stops drifting.

The application's own configuration

PART 3 · ONE FILE, READ BY TWO PROGRAMS

```
$ cp .env.example .env ; chmod 600 .env      # private BEFORE you edit
DATABASE_URL="mysql://school:<password>@127.0.0.1:3306/school"
STUDENT_ID=...  PORT=3xxx  HOST=0.0.0.0
DATABASE_URL="mysql://school:${SCHOOL_PASSWORD}@..." ← DO NOT
```

- The password is in a file two directories away, so `${SCHOOL_PASSWORD}` is the obvious next thought.
- The Prisma CLI expands it. `systemd` does not — in Part 4 it hands over those eighteen characters, exactly as typed.
- It works every way you test it this afternoon, and fails the one way the application is run.

Second reason, standing either way: substitution cannot percent-encode — the next slide. `HOST=0.0.0.0` is a deliberate override too, and next week it goes back.

The password is in a URL

PART 3 · PERCENT-ENCODING

```
mysql:// USER : PASSWORD @ HOST : PORT / DATABASE
           ↑ a / in here ends the address early
```

Error: P1013 ... certain characters must be escaped

- A URL has a grammar, and your password sits in the middle of it. / ends the address; % is already an escape.*
- Measured over two hundred generated passwords: *one in five holds a / and breaks. The rest parse by luck.*
- Encode every time: @ and : survive this library, and the next one may split at the first @ instead.*
- Encode it ONCE. Encode an encoded password and %40 becomes %2540 — a wrong password, and no error that says so.*

Nothing here is Prisma, or Node, or MySQL. It is the URL grammar, and you meet it again in every connection string, callback URL and token you ever put in a query parameter.

The command everyone gives you

PART 3 · DEVELOPING IS NOT DEPLOYING

```
$ npx prisma migrate dev --name init    # a DEVELOPER's command  
Prisma Migrate could not create the shadow database.  
User was denied access on the database `prisma_migrate_shadow_db_...`
```

- It wants to WRITE a schema — and first builds a scratch copy of your database to do it safely.
- Your user cannot create databases. You decided that yourself in Part 2, correctly. Do not fix this with a bigger grant.
- And even with every privilege on the server it would still be wrong: you are not writing this schema. It exists. You built it from [schema.sql](#).
- Every tutorial on the internet gives you this line. Reading the error beats searching for one.

Two failures in two minutes, neither of them a fault: one says your URL is not a URL, this one says your least-privilege grant is doing its job.

Ask the question you actually have

PART 3 · DOES THE DATABASE MATCH THE CODE?

```
$ npx prisma migrate diff --exit-code \  
  --from-schema-datasource prisma/schema.prisma \  
  --to-schema-datamodel    prisma/schema.prisma  
No difference detected.      $ echo $? → 0      (2 = drift)
```

- A sentence: **FROM** what the database holds, **TO** what the code describes — the answer is the work to make them agree.
- **--exit-code** makes it more than something to look at — a number a script can act on at 3am with nobody reading
- The database is owned by the database scripts; the application is a client that checks its assumptions.

Run BEFORE starting an application, not after it has failed. Drift comes itemised. After class you meet the other flow, where nothing is there yet and a THIRD `migrate` subcommand builds it — finding out which one is part of that lab.

4

Run it as a service

This is class 2's heartbeat again, except that what you are handing to systemd is a real application with a database behind it. Nothing new is needed, and that is the point.

A unit file you can already write

PART 4 · SYSTEMD

```
User=sysadmin                # NOT root
WorkingDirectory=/srv/school/backend
EnvironmentFile=/srv/school/backend/.env
ExecStart=/usr/bin/node server.js    Restart=always    RestartSec=5
NoNewPrivileges PrivateTmp ProtectSystem ProtectHome ← given to you
$ sudo systemctl daemon-reload ; sudo systemctl enable --now school-api
```

- A unit with no `User=` runs as root. Class 3's web server dropped privilege; yours must be told to.
- `WorkingDirectory=` is why `ExecStart=` can end in a bare filename. The interpreter still needs its absolute path.
- `EnvironmentFile=` names a FILE, and `systemd` — not your code — reads it. The same code runs in a container in class 8, supplied differently.

`RestartSec=` is the pause before retrying; without one a program that fails instantly is retried flat out. And `daemon-reload` is not decoration: edit the file without it and systemd answers from the version it read last — to you, and to the checker, which refuses to judge a stale unit.

PART 4 · SIXTY MACHINES, SIXTY NUMBERS

```
$ hostname  
lvm68042
```

→ **your API listens on 3042**

- 3, then the last three digits of your machine's name. Nobody else has your number.
- It goes in .env as PORT, not in the unit file — it is configuration, not code and not systemd's business.
- If the log names a port you did not choose, PORT never reached the program. Read the log, then look at the file.
- Your noticeboard keeps 3000. Two applications, two different numbers, and one machine that does not care.

Two programs cannot hold the same port. That is not a rule of this course, it is a rule of the machine — and you will meet it for real soon enough.

PART 4 · CURL IS THE HONEST VIEW

```
$ curl -s http://127.0.0.1:<your port>/api/health
$ curl -s -X POST -H 'Content-Type: application/json' \
    -d '{"studentId":"...", "firstName":"..."}' <url>
$ curl -s -i ... | head -1          # -i shows the status line
```

- Your backend course teaches what a REST API is. This one teaches how to verify one.
- 2xx it worked · 4xx YOUR request was wrong · 5xx the SERVER broke.
- That is the first thing to establish in an outage: it says whose problem it is before anyone starts arguing.
- Send an incomplete request. Send the same one twice. Send a DELETE twice. Four situations, four answers, none of them a bug.

`curl` enforces nothing a browser would. Remember that sentence — next week it costs somebody an hour.

Predict, then go and find out

PART 4 · THE POINT OF THE WHOLE CLASS

```
$ curl -X POST ... /api/students      # put yourself on the register  
$ sudo systemctl restart school-api  # kill the program. start it again.  
$ curl -s ... /api/students          # ?
```

- Say what you expect before you run the third line.
- The program that accepted you is gone: different process, different pid, no memory of anything.
- Whatever you observe, say WHY in one sentence. That sentence is what a database is for.
- In class 3 the Add button could not work at all. It works now, and what it accepts outlives the program.

Ten seconds to run, and it is the whole difference between a website and an application.

PART 4 · YOU HAVE DONE THIS BEFORE

- Your API is listening. From your laptop, it is not there. This is class 3, Part 3, unchanged.
- `Process · port · path` in. Three questions, three different failures, and telling them apart IS the diagnosis.
- Local check passes, my probe fails — that is the firewall, every time, and nothing else looks like it.
- Go and count the doors you have opened on this machine, and be able to say what each is for. Next week you close one on purpose and nothing breaks — which is the argument for class 5.

Every port you open is a promise to keep watching it. The best number of open ports is the smallest one that works.

LAB04 · THE MARKED ONE

- The same procedure again, on your noticeboard, on your own.* Due 30 August, 23:59.
- It needs: *its own database and user, its own generated password, its own configuration file, its own migration applied, its own unit file, and port 3000.*
- Seven written answers. Two of them need today's deployment still on your machine, so do not tidy it away.*
- The sheet gives you the tasks and not the commands.* Today's sheet keeps the explanations — go back to it as often as you like.
- Missed today? lab04class has no deadline and no mark. Do it first.

Looking a command up and typing it again is the part that makes it stick. That is the only reason the second sheet is thin.

Before you leave today

THE FOUR THAT MATTER

- Your API answers from a machine that is not your VM, and reports your own student number.
- You put yourself on the register, restarted the service, and can say in one sentence why the result is what it is.
- You can say what your database user may do, and what it may not.
- You read at least two error messages on purpose rather than by accident.
- If the service will not start: `systemctl status`, then `journalctl -u school-api`. The answer is in there, in English, most of the time.

Deadline and section dates are in the sheet. Ask me before you leave rather than at 23:00.